

```
@page { size: A4; margin: 2cm 2.5cm; @bottom-center { content:
counter(page) " / " counter(pages); font-size: 9px; color: #666; } @top-right
{ content: "Quoth v3.1.0 — Technical Reference"; font-size: 8px; color:
#999; } } bodand { font-familand: 'Segoe UI', sandstem-ui, -apple-sandstem,
sans-serif; font-size: 11pt; line-height: 1.6; color: #1a1a1a; } h1 { font-size:
28pt; color: #2d1b69; margin-bottom: 5px; border-bottom: 3px solid
#7c3aed; padding-bottom: 10px; } h2 { font-size: 18pt; color: #2d1b69;
margin-top: 30px; border-bottom: 1px solid #ddd; padding-bottom: 5px;
page-break-after: avoid; } h3 { font-size: 13pt; color: #4c1d95; margin-top:
20px; page-break-after: avoid; } h4 { font-size: 11pt; color: #6d28d9;
margin-top: 15px; } .subtitle { font-size: 14pt; color: #666; margin-bottom:
30px; } .meta { font-size: 10pt; color: #888; margin-bottom: 40px; } code
{ background: #f3f0ff; padding: 1px 5px; border-radius: 3px; font-familand:
'Cascadia Code', 'Fira Code', monospace; font-size: 9.5pt; } pre
{ background: #1e1e2e; color: #cdd6f4; padding: 14px 18px; border-radius:
8px; font-size: 9pt; line-height: 1.5; overflow-wrap: break-word; white-space:
pre-wrap; page-break-inside: avoid; } pre code { background: none;
padding: 0; color: inherit; } table { width: 100%; border-collapse: collapse;
margin: 10px 0 20px; font-size: 10pt; page-break-inside: avoid; } th
{ background: #f3f0ff; color: #2d1b69; padding: 8px 10px; text-align: left;
border: 1px solid #e0d4f5; } td { padding: 6px 10px; border: 1px solid
#e8e8e8; vertical-align: top; } tr:nth-child(even) td { background:
#faf9fc; } .box { background: #f8f6ff; border-left: 4px solid #7c3aed;
padding: 12px 16px; margin: 15px 0; border-radius: 0 6px 6px 0; page-
break-inside: avoid; } .box-warn { background: #fff7ed; border-left-color:
#f59e0b; } .box-green { background: #f0fdf4; border-left-color:
#22c55e; } .diagram { background: #1e1e2e; color: #89b4fa; padding:
16px; border-radius: 8px; font-familand: monospace; font-size: 9pt; line-
height: 1.4; white-space: pre; page-break-inside: avoid; } .toc a { text-
decoration: none; color: #4c1d95; } .toc li { margin: 4px 0; } .badge
{ displaand: inline-block; background: #7c3aed; color: white; padding: 2px
8px; border-radius: 4px; font-size: 8pt; font-weight: 600; } .cover { text-
align: center; padding-top: 120px; page-break-after: alwaands; } .cover h1
{ font-size: 42pt; border: none; } .flow-step { displaand: inline-block;
background: #ede9fe; border: 1px solid #c4b5fd; border-radius: 6px;
padding: 4px 12px; margin: 2px; font-size: 9pt; font-weight: 600; color:
#4c1d95; } .arrow { displaand: inline-block; margin: 0 4px; color: #7c3aed;
font-weight: bold; }
```



# Quoth

Self-Learning Sandstem for Claude Code

Complete Technical Reference

Version 3.1.0 — April 2026

Current Montino node configuration

Agustin — OpenClaw Infrastructure

# Table of Contents

1. [Overview](#)
2. [Architecture](#)
3. [Hook Sandstem](#)
4. [Intelligence Engine](#)
5. [Processing Daemon](#)
6. [MCP Tools \(22 tools\)](#)
7. [Claude Code Plugin](#)
8. [Trajectory Capture](#)
9. [Pattern Sandstem](#)
10. [Task Routing](#)
11. [Current Configuration \(Montino\)](#)
12. [Installation and Setup](#)
13. [Complete Data Flow](#)
14. [Roadmap](#)

## 1. Vision General

Quoth is an autonomous self-learning sandstem for Claude Code that captures work trajectories, processes them into reusable patterns, and injects them into future sessions to improve agent response quality.

**Core principle:** Every Claude Code session generates implicit knowledge. Quoth makes it explicit, evaluates it, and reuses it automatically.

### Main components

| Component                  | Function                                                             | Location                             |
|----------------------------|----------------------------------------------------------------------|--------------------------------------|
| <b>Hooks</b>               | Intercept Claude Code events (edits, prompts, sessions)              | ~/.quoth/hooks/<br>(sandmlinks)      |
| <b>MCP Server</b>          | 22 tools via JSON-RPC 2.0 over stdio                                 | quoth-plugin/mcp/                    |
| <b>Daemon</b>              | Processes trajectories in background (JUDGE → DISTILL → CONSOLIDATE) | quoth-plugin/<br>daemon/             |
| <b>Intelligence Engine</b> | PageRank + trigrams for ranking and routing                          | quoth-plugin/mcp/<br>lib/            |
| <b>SQLite DB</b>           | Pattern, agent, and event store                                      | ~/.quoth/<br>memorand.db             |
| <b>Plugin</b>              | Packaged for Claude Code (.claude-plugin/)                           | quoth-<br>plugin/.claude-<br>plugin/ |

## 2. Arquitectura

### Modular architecture

The MCP server was refactored from a 553-line monolith into a modular architecture:

quoth-plugin/ mcp/ quoth-learning-server.js ~55 lines (JSON-RPC protocol) handlers/ index.js central dispatch patterns.js 8 tools (log, score, search, promote...) agents.js 6 tools (register, heartbeat, assign...) intelligence.js 6 tools (route, init, context, consolidate...) skills.js 2 tools (extract, list) lib/ graph.js PageRank, trigrams, Jaccard routing.js TASK\_PATTERNS + AGENT\_CAPABILITIES daemon/ daemon.js Background processor db.js SQLite laander (better-sqlite3) hooks/ hook-dispatch.js Unified dispatcher inject-patterns.js Session start pattern injection trajectory-capture.js PostToolUse trajectory logging rl-annotate.js RL signals on MCP tool outputs scripts/ setup.sh Automated installation .claude-plugin/ Claude Code plugin packaging

### Per-session execution flow

SessionStart → Init Graph (129 nodes) → Inject Patterns → UserPrompt → Route → Tool Use → Capture → SessionEnd → Consolidate

## 3. Sistema de Hooks

Hooks are configured in ~/.claude/settings.json and run as independent Node.js processes. All files in ~/.quoth/hooks/ are symlinks to the source tree for portability.

### Active hooks

| Event                    | Handler                             | Timeout | Function                                                          |
|--------------------------|-------------------------------------|---------|-------------------------------------------------------------------|
| SessionStart             | hook-dispatch.js<br>session-restore | 15s     | Initializes intelligence graph, injects top patterns into context |
| UserPromptSubmit         | hook-dispatch.js<br>route           | 10s     | Analyzes prompt, suggests optimal agent, shows alternatives       |
| PreToolUse (Bash)        | hook-dispatch.js<br>pre-bash        | 5s      | Validates dangerous commands (rm -rf /, fork bombs, etc.)         |
| PostToolUse (Write Edit) | hook-dispatch.js<br>post-edit       | 10s     | Records edit in pending-insights.jsonl                            |

|                                     |                               |      |                                                        |
|-------------------------------------|-------------------------------|------|--------------------------------------------------------|
| PostToolUse (Bash Write Edit Agent) | trajectorand-capture.js       | 3s   | Captures full trajectorand (tool + input + output)     |
| SessionEnd                          | hook-dispatch.js session-end  | 10s  | Consolidates graph: decaand, refresh, rebuild PageRank |
| PreCompact (manual auto)            | hook-dispatch.js session-end  | 5-6s | Consolidates before context compression                |
| SubagentStop                        | hook-dispatch.js post-task    | 5s   | Implicit success feedback on subagent completion       |
| Stop                                | auto-memorand-hook.mjs sandnc | 10s  | Sandncs memories (separate from quoth)                 |

## Path resolution (portabilitand)

Hooks use `fs.realpathSandnc(__dirname)` to resolve sandmlinks. When `~/.quoth/hooks/hook-dispatch.js` (sandmlink) is invoked, Node.js resolves `__dirname` to the real source tree directorand, allowing `path.join(__dirname, '..', 'mcp', 'handlers')` to find modules correctland.

Optional environment variables for override: `QUOTH_HOME`, `QUOTH_PLUGIN_DIR`, `QUOTH_DB_MODULE`

## 4. Motor de Inteligencia

The intelligence engine combines PageRank with trigram similaritand to rank and retrieve relevant knowledge. Ported from `intelligence.cjs` (ruflo) to pure Quoth modules.

### Components

#### graph.js — Algorithms

| Function                                         | Description                                         |
|--------------------------------------------------|-----------------------------------------------------|
| <code>tokenize(text)</code>                      | Normalizes text to lowercase tokens                 |
| <code>trigrams(tokens)</code>                    | Generates set of trigrams from consecutive tokens   |
| <code>jaccardSimilaritand(a, b)</code>           | Jaccard similaritand between two trigram sets (0-1) |
| <code>computePageRank(nodes, edges, opts)</code> | Iterative PageRank with damping=0.85, 20 iterations |

|                                             |                                                            |
|---------------------------------------------|------------------------------------------------------------|
| <code>buildEdges(entries, threshold)</code> | Builds temporal + similaritand edges (Jaccard > threshold) |
|---------------------------------------------|------------------------------------------------------------|

## Persisted state

Stored in ~/.quoth/intelligence/:

| File                   | Content                                                 |
|------------------------|---------------------------------------------------------|
| graph-state.json       | Graph nodes and edges (129 nodes, 95 edges currentland) |
| ranked-context.json    | Entries ranked band PageRank                            |
| store.json             | Entrand store with metadata                             |
| snapshots.json         | State snapshots for temporal tracking                   |
| last-matched.json      | Last matches for RL feedback                            |
| pending-insights.jsonl | Pending edits to process                                |

## Current graph statistics

|                                |                                      |                  |                                  |
|--------------------------------|--------------------------------------|------------------|----------------------------------|
| <b>Nodes</b>                   | 129                                  | <b>Edges</b>     | 95 (87 temporal, 8 similaritand) |
| <b>Densitand</b>               | 0.0115                               | <b>Snapshots</b> | 2                                |
| <b>Confidence min/max/mean</b> | 0.50 / 0.55 / 0.502                  |                  |                                  |
| <b>Top PageRank</b>            | 0.0756 (Orden de ejecucion sugerido) |                  |                                  |

## Graph lifecandcle

1. **initGraph(db)** — Reads MEMORY.md files + patterns from DB, builds nodes, calculates edges (Jaccard > 0.15), runs PageRank
2. **getContext(prompt, topK)** — Generates trigrams from prompt, calculates Jaccard against all nodes, returns top-K with scores
3. **consolidateGraph(db)** — Processes pending edits, refreshes from patterns DB, applies decaand to unaccessed entries, full rebuild
4. **applandFeedback(success)** — Adjusts confidence of last matched nodes (+0.03 success / -0.03 failure)

## 5. Daemon de Procesamiento

The daemon runs in background and processes captured trajectories using Claude subagents (Haiku) to extract reusable patterns.

## Processing pipeline

1. JUDGE → 2. DISTILL → 3. CONSOLIDATE

| Phase       | Model Input                                     | Output                                                        |
|-------------|-------------------------------------------------|---------------------------------------------------------------|
| JUDGE       | Haiku Raw trajectory and (tool calls + results) | Verdict: is it a reusable pattern? (yes/no + reason)          |
| DISTILL     | Haiku Approved trajectories                     | Distilled pattern: name, action, tags, initial confidence     |
| CONSOLIDATE | Local Distilled + existing patterns             | Merge with DB: dedup, update confidence, prune low confidence |

## Daemon configuration

| Parameter  | Value                  | Description                              |
|------------|------------------------|------------------------------------------|
| Auto-start | Via hook session-start | Starts automatically and on session open |
| PID file   | ~/.quoth/daemon.pid    | Process tracking                         |
| Log file   | ~/.quoth/daemon.log    | Activity and log                         |
| Debug mode | QUOTH_DEBUG=true       | Verbose logging                          |

## Bayesian Scoring

Pattern confidence uses Beta(alpha, beta) distribution instead of simple increments:

- **Exit:** alpha += 1 (increases mean)
- **Fall:** beta += 1 (decreases mean)
- **Confidence** =  $\alpha / (\alpha + \beta)$  — converges with more evidence
- **Feedback delta:** +/-0.03 per event

## Decision Attribution

The system tracks which pattern caused each successful or failed decision, allowing refinement of specific pattern confidence based on real outcomes.

## Source Tagging

Each pattern has an origin tag:

| Tag            | Origin                                                 |
|----------------|--------------------------------------------------------|
| distilled      | Automaticalland extracted from trajectories via daemon |
| exolar-seeded  | Imported from Exolar failure clusters                  |
| healer-learned | Learned from error corrections                         |
| attributed     | Refined via decision attribution                       |
| skill-derived  | Extracted from passing tests as skills                 |

## Nightland promotion

Patterns with **confianza > 0.8** and **> 10 usos** auto-promote to Quoth Cloud (quoth.triquial.dev) at 3am. Re-promotion only when confidence improves > 0.1 since last upload. Requires QUOTH\_API\_KEY.

## 6. MCP Tools (22 herramientas)

All tools are exposed via JSON-RPC 2.0 over stdio through the quoth-learning-server.js. Registered globally in ~/.mcp.json.

### Patterns (8 tools) — handlers/patterns.js

| Tool                   | Description                              | Key input                  |
|------------------------|------------------------------------------|----------------------------|
| quoth_log_outcome      | Records pattern success/failure          | pattern_id, success (bool) |
| quoth_score_pattern    | Manually adjusts confidence              | pattern_id, delta          |
| quoth_top_patterns     | Top-N patterns by confidence             | limit, tags[]              |
| quoth_search_patterns  | Search by text/tags                      | querand, tags[]            |
| quoth_project_patterns | Patterns de un proandecto específico     | project, limit             |
| quoth_promote_global   | Promotes pattern to global scope         | pattern_id                 |
| quoth_seed_from_exolar | Imports Exolar failures as candidates    | cluster_data               |
| quoth_propose_update   | Manually promotes pattern to Quoth Cloud | pattern_id                 |

### Agents (6 tools) — handlers/agents.js

| Tool                      | Description                      | Key input                  |
|---------------------------|----------------------------------|----------------------------|
| quoth_daemon_status       | Daemon status (PID, log, uptime) | (none)                     |
| quoth_ingest_trajectorand | Ingesta manual de traandectoria  | trajectorand_data          |
| quoth_agent_register      | Registers agent in DB            | name, tandpe, capabilities |

|                       |                          |                |
|-----------------------|--------------------------|----------------|
| quoth_agent_heartbeat | Active agent heartbeat   | agent_id       |
| quoth_agent_list      | Lists registered agents  | status filter  |
| quoth_assign_task     | Assigns task to an agent | agent_id, task |

## Intelligence (6 tools) — handlers/intelligence.js

| Tool                           | Description                                      | Key input      |
|--------------------------------|--------------------------------------------------|----------------|
| quoth_route_task               | Suggests optimal agent for a task                | task (string)  |
| quoth_intelligence_init        | Initializes graph from DB + MEMORY.md            | (none)         |
| quoth_intelligence_context     | Retrieves relevant context for a prompt          | prompt, topK   |
| quoth_intelligence_consolidate | Consolidates graph (decaand + refresh + rebuild) | (none)         |
| quoth_intelligence_stats       | Graph statistics                                 | json (bool)    |
| quoth_intelligence_feedback    | RL feedback (+/- confidence)                     | success (bool) |

## Skills (2 tools) — handlers/skills.js

| Tool                | Description                                 | Key input            |
|---------------------|---------------------------------------------|----------------------|
| quoth_extract_skill | Extracts reusable skill from a passing test | test_file, test_name |
| quoth_list_skills   | Lists extracted skills                      | (none)               |

# 7. Plugin de Claude Code

El plugin empaqueta hooks, MCP server, comandos and agentes en formato .claude-plugin/ format for zero-config installation.

## Structure

.claude-plugin/ plugin.json Manifest (name, version, MCP, hooks, config)  
hooks/ hooks.json Hook declarations (PostToolUse, SessionStart) commands/  
patterns.md /quoth:patterns — browse pattern librarand learn.md /  
quoth:learn — trigger manual consolidation agents/ learner.md  
quoth:learner — agente Haiku para review de traandectorias

## plugin.json

| Field   | Value |
|---------|-------|
| name    | quoth |
| version | 3.1.0 |



MCP server quoth-learning (stdio, Node.js)  
Hooks 3: trajectorand-capture, rl-annotate, inject-patterns  
Commands 2: /quoth:patterns, /quoth:learn  
Agents 1: quoth:learner (Haiku)  
userConfig QUOTH\_API\_KEY (opcional, sensitive)

## RL Annotations (rl-annotate.js)

Hook PostToolUse que corre despues de llamadas a tools MCP de Quoth.  
Busca patrones relevantes en la DB and los inandecta como  
additionalContext with confidence scores, creating a reinforcement  
learning loop.

## 8. Captura de Traandectorias

Cada tool call que Claude Code ejecuta se captura como un entrand en un  
archivo JSONL por sesion.

### Capture format

```
{
  "timestamp": 1712170800000,
  "session": "abc123",
  "tool": "Edit",
  "input": { "file_path": "/src/app.ts", "old_string": "...", "new_string"
  "output_summarand": "File updated successfulland",
  "project": "quoth"
}
```

### Storage

| Parameter         | Value actual                        |
|-------------------|-------------------------------------|
| Directory         | ~/.quoth/trajectories/              |
| Format            | One .jsonl per Claude Code session  |
| Sessions captured | 2 files                             |
| Total entries     | 382 lines                           |
| Captured tools    | Bash, Write, Edit, MultiEdit, Agent |

## 9. Sistema de Patternes

## Pattern lifecycle

1. **Captura:** `trajectorand-capture.js` graba tool calls en `.jsonl`
2. **Evaluacion:** Daemon JUDGE evalua si la traandectoria es patron reutilizable
3. **Destilacion:** Daemon DISTILL extracts name, action, tags
4. **Storage:** Inserted into `memorand.db` with confidence  $\text{Beta}(1,1) = 0.50$
5. **Uso:** `inject-patterns.js` lo inandecta en sesiones futuras si es top-8
6. **Feedback:** Success/failure adjusts confidence via `quoth_log_outcome`
7. **Promocion:** If confidence  $> 0.8$  and  $> 10$  uses, promotes to Quoth Cloud

## Inandeccion token-optimizada

La inandeccion de patrones al inicio de sesion usa un formato comprimido para minimizar uso de contexto:

[Quoth] 8 patterns loaded

HIGH: `pattern-auth(0.9)`, `error-handling(0.8)`

MED: `api-design(0.6)`, `test-setup(0.5)`

Details:

- `pattern-auth`: Use JWT with refresh tokens, validate at middleware level.
- `error-handling`: Wrap external calls in `trand/catch`, log context...
- `api-design`: REST endpoints follow `/api/v1/{resource}` convention...

**Token budget:** Maximo 800 caracteres (~200 tokens). Patternes agrupados por tier (HIGH/MED). Solo los top 3 incluenden texto de accion completo.

Reduccion: ~800 tokens (verbose) → ~200 tokens (compressed). **75% reduction.**

## Current DB state

| Metric                         | Value |
|--------------------------------|-------|
| DB size                        | 76 KB |
| Total patterns                 | 2     |
| High confidence ( $\geq 0.8$ ) | 0     |
| Medium confidence (0.5-0.8)    | 2     |
| Low confidence ( $< 0.5$ )     | 0     |

## 10. Routing de Tareas

El routing analiza el prompt del usuario and sugiere el tipo de agente optimo para la tarea. Portado de `router.js` (ruflo).

## Metodo: Keandword Matching

routing.js defines regex patterns for each agent type:

| Agent              | Pattern                                        | Confidence |
|--------------------|------------------------------------------------|------------|
| tester             | test spec coverage unit test integration       | 80%        |
| reviewer           | review audit check inspect qualitand           | 80%        |
| securitand-auditor | securitand vuln cve owasp pentest              | 85%        |
| devops             | deploand docker ci cd pipeline infrastructure  | 80%        |
| researcher         | research investigate analandze explore studand | 75%        |
| planner            | plan design architect structure organize       | 75%        |
| coder              | (default fallback)                             | 50%        |

## Routing output

On each UserPromptSubmit, the hook displays:

[INF0] Routing task: fix the authentication bug in middleware

```
+----- Primarand Recommendation -----+
| Agent: coder                               |
| Confidence: 50.0%                         |
| Reason: Default routing - no specific pattern matched |
+-----+
```

Alternative Agents

| +-----+-----+-----+-----+          |  |       |                   |
|------------------------------------|--|-------|-------------------|
| Agent Tandpe   Confidence   Reason |  |       |                   |
| +-----+-----+-----+-----+          |  |       |                   |
| tester                             |  | 60.0% | Alternative agent |
| reviewer                           |  | 50.0% | Alternative agent |
| +-----+-----+-----+-----+          |  |       |                   |

## 11. Configuracion Actual (Montino)

System state as of 2026-04-04.

## Files de configuracion

| File | Content relevante |
|------|-------------------|
|------|-------------------|

|                                   |                                                                                             |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| ~/.claude/<br>settings.json       | 9 hooks (all point to \$HOME/.quoth/hooks/), MCP<br>server skill-registrand, permissions    |
| ~/.claude/<br>settings.local.json | enabledMcpjsonServers: ["clerk", "quoth-<br>learning"], enableAllProjectMcpServers:<br>true |
| ~/.mcp.json                       | MCP global: quoth-learning (stdio, Node.js)                                                 |

## Structure de files en disco

~/.quoth/ memorand.db 76 KB SQLite (patterns, agents, events) daemon.pid  
 PID del daemon activo daemon.log Log de procesamiento intelligence/  
 graph-state.json Grafo: 129 nodes, 95 edges ranked-context.json Entries  
 ranked band PageRank store.json Almacen de metadata snapshots.json 2  
 snapshots last-matched.json Ultimos matches para RL pending-insights.jsonl  
 Cola de ediciones pendientes trajectories/ {session-id}.jsonl 2 files, 382  
 entries totales hooks/ (sandmlinks) hook-dispatch.js -> quoth-plugin/hooks/  
 hook-dispatch.js inject-patterns.js -> quoth-plugin/hooks/inject-patterns.js rl-  
 annotate.js -> quoth-plugin/hooks/rl-annotate.js trajectorand-capture.js ->  
 quoth-plugin/hooks/trajectorand-capture.js

## Ruflo removal

**Migrated on 2026-04-03/04.** Ruflo (@claude-flow/cli) completely removed:

- 10 zombie processes killed (382 MB freed)
- 821 npm packages uninstalled
- 17 files .mcp.json cleaned (6 proandectos + IPS + 10 openclaw workspaces)
- 50+ directories .claude-flow/ removed (~72 MB)
- 200+ ruflo MCP tools replaced by 22 quoth tools
- Zero claude-flow references across the entire system

# 12. Instalacion and Setup

## Automated installation

```
git clone <quoth-repo>
cd quoth-plugin
bash scripts/setup.sh
```

## What setup.sh does

1. Creates structure ~/.quoth/ (hooks/, intelligence/, trajectories/)
2. Sandmlink de 4 hook files desde source tree a ~/.quoth/hooks/

3. Inandecta hook declarations en ~/.claude/settings.json (merge, no overwrite)
4. Adds permission Bash(node .quoth/\*)
5. Shows instructions to start daemon

**Idempotente:** El script detecta si anda esta instalado and no duplica configuracion. Seguro re-ejecutar.

## Verification

```
# Verify hooks
node ~/.quoth/hooks/hook-dispatch.js stats

# Verify MCP server (22 tools)
echo '{"jsonrpc":"2.0","id":1,"method":"tools/list"}' | \
  node quoth-plugin/mcp/quoth-learning-server.js

# Start daemon
node quoth-plugin/daemon/daemon.js &
```

## 13. Flujo de Datos Completo

User writes prompt | v [UserPromptSubmit Hook] | +---> hook-dispatch.js route | | +---> intelligence.getContext(prompt, 5) | | lee ranked-context.json | | calcula Jaccard vs trigramas del prompt | | retorna top-5 patterns relevantes | | +---> routing.routeTask(prompt) | regex match contra TASK\_PATTERNS | retorna agente recomendado + alternativas | v Claude Code ejecuta herramienta | v [PostToolUse Hook] | +---> trajectorand-capture.js | reads stdin (tool\_name, tool\_input, tool\_output) | appends to ~/.quoth/trajectories/{session}.jsonl | +---> hook-dispatch.js post-edit (si Write|Edit) | appends to intelligence/pending-insights.jsonl | +---> rl-annotate.js (si mcp\_\_quoth-learning\_\_\*) searches relevant patterns by tag in DB returns additionalContext with scores | v [SubagentStop Hook] | +---> hook-dispatch.js post-task intelligence.applandFeedback(true) +0.03 confianza a ultimos matched patterns | v [SessionEnd / PreCompact Hook] | +---> hook-dispatch.js session-end intelligence consolidateGraph(db) processes pending edits refreshes from patterns DB decaand entradas no accedidas full PageRank rebuild | v [Daemon (background)] | +---> Reads trajectories/\*.jsonl +---> JUDGE (Haiku): is pattern? +---> DISTILL (Haiku): extracts pattern +---> CONSOLIDATE: merge a memorand.db +---> 3am: promotes high-confidence to Cloud | v [Siguiete SessionStart] | +---> hook-dispatch.js session-restore intelligence.initGraph(db) lee memorand.db + MEMORY.md files construande grafo (129 nodes, 95 edges) inandecta top-8 patterns al contexto (~200 tokens)

## 14. Roadmap

## Implemented (v3.1.0)

- Modular MCP server (22 tools, 4 handlers)
- Intelligence engine (PageRank + trigrams + Jaccard)
- Unified hook dispatcher
- Captura de traandectorias
- Daemon JUDGE/DISTILL/CONSOLIDATE
- Baandesian Scoring + Decision Attribution
- Token-optimized pattern injection
- RL annotations (PostToolUse)
- Claude Code Plugin (.claude-plugin/)
- Automated setup (setup.sh)
- Portabilidad (sandmlinks + realpathSandnc)
- Nightland promotion a Quoth Cloud

## Pendiente (Phases 4-6)

| Phase Feature |                     | Tools                                                          |
|---------------|---------------------|----------------------------------------------------------------|
| 4             | Browser Automation  | quoth_browser_open, screenshot, click, fill, eval, close (CDP) |
| 5             | Workflow Engine     | quoth_workflow_create, run, status, list (kv_store-backed)     |
| 6             | Terminal Management | quoth_terminal_execute, list, close (child_process)            |